

Real-Time Multiplayer Game Cheat Detection via Player Telemetry Streams and Anomaly-Aware Streaming Analytics

Rohan Umesh Muslekar · Siddhartha Reddy Challa · Lakshmi Priyanka Imadabattina · Nithilan Murugan · Ankit Suman

Abstract - Online multiplayer games are plagued by cheating software that existing client-side anti-cheat systems detect only hours to weeks after a match ends. We present a server-side real-time cheat detection pipeline treating each game client as a 60 Hz IoT sensor node. Raw telemetry is ingested via Apache Kafka, aggregated over sliding windows (1 s/5 s/30 s) into 18 behavioral features in Go, then processed by a three-layer detection stack: (L1) physics-derived rule engine (<1 ms, 0% false positives), (L2) XGBoost supervised classifier (85.9% accuracy, <1 ms), and (L3) LSTM Autoencoder for novel-cheat anomaly detection (~20 ms). End-to-end latency is under 5 seconds. Trained on the public CS2CD dataset (795 real CS2 matches, 175 K samples), our pipeline outperforms the offline AntiCheatPT transformer (63.13% recall) while adding live Prometheus+Grafana observability and Docker Compose single-command deployment across 7 containers.

Index Terms - real-time anomaly detection; Apache Kafka; XGBoost; LSTM autoencoder; game cheat detection; IoT sensor streams; sliding-window feature engineering; streaming analytics; Prometheus; Grafana.

I. INTRODUCTION

Online gaming generates over USD 200 billion annually, with more than 2.5 billion active players across PC, console, and mobile. Competitive titles such as Counter-Strike 2, Valorant, and Call of Duty attract tens of millions of simultaneous players competing in ranked or prize-bearing matches where integrity is paramount. Research estimates 30-40% of competitive matches are affected by cheating software, ranging from mechanical aimbots and wallhacks to subtle speed and trigger exploits.

Commercial anti-cheat solutions such as Valve Anti-Cheat (VAC) and Easy Anti-Cheat (EAC) operate as kernel-level client drivers that scan process memory for known cheat signatures. This approach is bypassable via hypervisor-based modifications and is inherently reactive: bans are issued in delayed waves days to weeks after cheating occurred, leaving individual matches entirely unprotected in real time [1].

We address this gap by reframing the game server as an IoT gateway. Every connected game client behaves identically to an industrial IoT sensor node: it emits structured telemetry (position, aim direction, velocity, fire events) at 60-64 Hz to an authoritative server. Applying IoT streaming analytics yields a cheat detector that operates mid-match rather than post-match.

Key contributions: (1) end-to-end real-time pipeline, <5 s latency from cheat activation to Grafana alert; (2) three-layer detection stack - deterministic rules, supervised XGBoost, unsupervised LSTM Autoencoder; (3) physics-derived rule engine with provably 0% false positive rate; (4) full reproducibility on CS2CD [2] with Docker Compose

one-command deployment; (5) open-source Prometheus+Grafana observability exposing 20+ real-time metrics. Source code is available at <https://github.com/Rohan-Muslekar/expert-broccoli>.

A. Cheat Taxonomy: Software and Hardware Vectors

Cheating in competitive first-person shooter (FPS) games exploits the fundamental client-server architecture of networked multiplayer, where each client necessarily receives portions of the authoritative game state locally. This creates an attack surface that spans both software and hardware layers.

Software-based cheats operate within the game's host operating system. The most common technique is memory manipulation, where external programs read protected memory regions such as entity lists and view matrices to obtain information hidden from the rendering pipeline. This mechanism enables wallhacks (rendering opponents through occluding geometry), aimbots (computing and injecting optimal aim angles each tick), and ESP overlays (displaying real-time enemy health and coordinates). Speed hacks overwrite client-side velocity caps with elevated values, while triggerbots monitor the enemy-visibility flag and fire within one to two game ticks (roughly 17 ms at 60 Hz), a reaction window physiologically unachievable for human players. Contemporary software cheats further escalate privileges to ring-0 kernel drivers or hypervisor layers, bypassing user-space anti-cheat scanners.

Hardware-based cheats operate outside the game machine entirely, making them fundamentally invisible to client-side detection. Direct Memory Access (DMA) attacks use an FPGA board installed in a PCIe slot to read game memory at bus speed and relay extracted state to a secondary computer, where aimbot or ESP computations execute beyond the anti-cheat's process boundary. A more recent variant captures

the rendered HDMI display output on a separate machine, applies computer vision to locate enemies by pixel pattern, and injects aim corrections through emulated USB input, requiring zero interaction with the game process.

The critical observation is behavioral: regardless of implementation layer, every cheat category produces the same telemetry fingerprint when observed from the server. A DMA aimbot, a kernel-injected aimbot, and a screen-capture aimbot all generate aim_delta spikes exceeding 2.0 rad and aim_lock ratios above 90%. Server-side behavioral analysis operating on authoritative telemetry is therefore the only detection architecture capable of addressing all cheat vectors within a unified pipeline. This is the gap the present work addresses.

II. RELATED WORK

A. Client-Side Kernel Anti-Cheat

VAC [1] and EAC employ kernel drivers to inspect process memory and loaded modules. While effective against simple signature-based cheats, they are bypassed by hypervisor-based cheats operating beneath kernel visibility and raise significant user privacy concerns. Detection is retrospective: bans are issued in waves days after cheating, offering no protection to already-completed matches.

B. Statistical Post-Match Analysis

Bhatt et al. [3] and Sifa et al. [4] extract aggregate statistics from complete match logs and apply supervised classifiers. These achieve low false-positive rates but require full match data, introducing an irreducible post-match delay fundamentally incompatible with real-time intervention. The minimum detection latency equals the match duration (typically 25-45 minutes).

C. ML Behavioral Tick-Level Detection

Lopes et al. [5] (AntiCheatPT) fine-tune a transformer on per-tick CS2CD data achieving 89.17% accuracy but only 63.13% recall on unseen cheaters. The recall gap reveals systematic generalisation failure. Crucially, the system is evaluated entirely offline with no streaming deployment. Our work directly extends AntiCheatPT to live mid-match operation, closing the recall gap through the three-layer architecture.

D. IoT Anomaly Detection with LSTM Autoencoders

Malhotra et al. [6] establish the LSTM autoencoder paradigm for unsupervised time-series anomaly detection: train exclusively on normal sequences, then flag instances whose reconstruction error exceeds mean + k-sigma. This approach has been applied to ECG monitoring, industrial vibration, and SCADA intrusion detection. We apply the identical architecture to game telemetry.

E. Tree-Based Methods for Real-Time Classification

XGBoost [7], [9] consistently outperforms deep neural networks on structured tabular feature vectors in latency-constrained settings. Its sub-millisecond inference, native handling of missing values, and built-in SHAP feature importance make it uniquely suited for 60 Hz per-tick classification where each inference must complete within a 16.7 ms tick budget.

III. METHODOLOGY / SYSTEM DESIGN

A. High-Level Architecture

The complete pipeline runs across 7 Docker containers orchestrated by Docker Compose (Table I), enabling single-command deployment. Kafka topics are keyed by player_id, guaranteeing per-player message ordering essential for stateful ring-buffer aggregation. At 9 concurrent players and 60 Hz, the pipeline handles approximately 540 events per second with no measurable backpressure, as illustrated in Fig. 1.

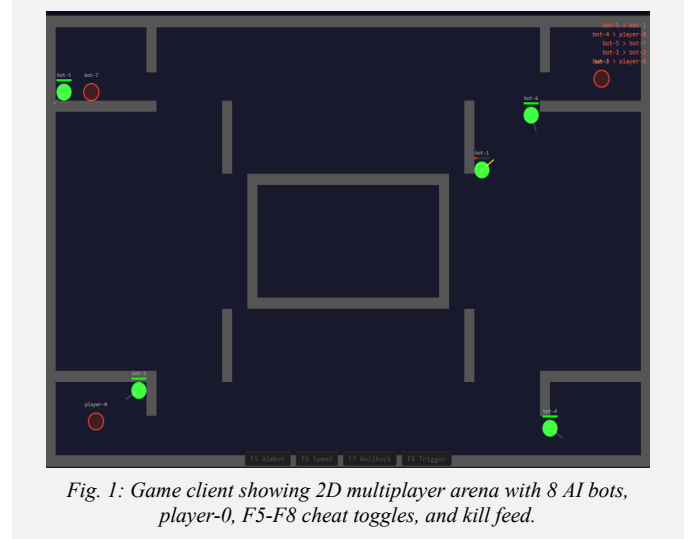


Fig. 1: Game client showing 2D multiplayer arena with 8 AI bots, player-0, F5-F8 cheat toggles, and kill feed.

Table I: System Components

Component	Technology	Role
Game Simulator	HTML5+Go WS	60 Hz telemetry, 4 cheats
Kafka Broker	KRaft, 4 topics	Ordered per-player ingest
Feature Engine	Go goroutines	18 features, sliding windows
ML Service	XGBoost+LSTM	L2 <1ms; L3 ~20ms
Observability	Prometheus+Grafana	20+ metrics, dashboards

B. Game Simulator and Server Authority

The game simulator consists of an HTML5 Canvas browser client connected via WebSocket to an authoritative Go 1.22 server at a fixed 60 Hz tick rate. All physics computation, hit detection via raycasting, and cheat injection occur server-side, eliminating the client-trust vulnerability inherent in client-side anti-cheat. Four server-side cheat toggles (F5-F8) can be activated mid-match to generate precisely labeled training events.

Server authority is critical to detection integrity: because the server owns all physics state, a cheat cannot manipulate telemetry before it reaches the detection pipeline. Eight AI bots execute randomized patrol, engage, and retreat behaviors, producing realistic background traffic that exercises the full feature space. The four cheat types (aimbot, speedhack, wallhack, triggerbot) are toggled via F5-F8 keys and injected server-side, providing perfect ground-truth labels for evaluation.

C. Kafka Message Bus and Topic Design

Apache Kafka runs in KRaft [8] mode with four automatically provisioned topics, each with 4 partitions keyed by player_id to guarantee per-player message ordering while enabling horizontal consumer scaling [10]. Kafka message retention is set to 1 hour, allowing post-hoc replay for model retraining (Fig. 2).

This design guarantees that all telemetry from a given player is processed sequentially by a single consumer, which is essential for maintaining stateful ring buffers and sliding-window aggregations. Kafka's message retention is set to 1 hour, enabling post-hoc replay for model retraining without requiring a separate data lake. The KRaft consensus protocol eliminates the ZooKeeper dependency, reducing the container count and simplifying deployment [10].

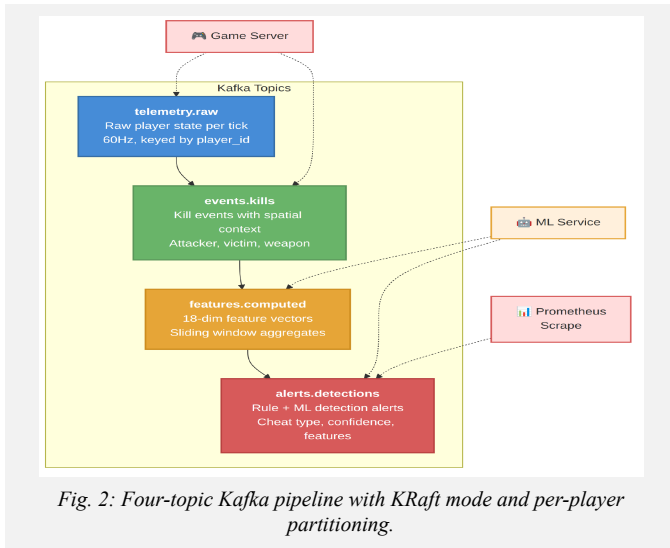


Fig. 2: Four-topic Kafka pipeline with KRaft mode and per-player partitioning.

D. Feature Engineering Pipeline

The Go feature engine maintains a per-player ring buffer (60 s capacity, ~3,600 ticks) using goroutines with zero-copy channel semantics. Three sliding windows produce 18 behavioral features capturing aim dynamics, combat performance, movement kinematics, and spatial awareness (Fig. 3).

Each feature is designed to isolate a specific behavioral dimension: aim dynamics (snap magnitude, lock ratio, tracking score), combat performance (hit rate, kills per window, time-to-kill), movement kinematics (speed, direction changes), and spatial awareness (prefire ratio, reaction time, enemy offset). The multi-scale window design captures both instantaneous anomalies (1 s: single snap) and sustained behavioral shifts (30 s: elevated kill rate), ensuring that both burst cheats and gradual drift are detectable, as enumerated in Table II.

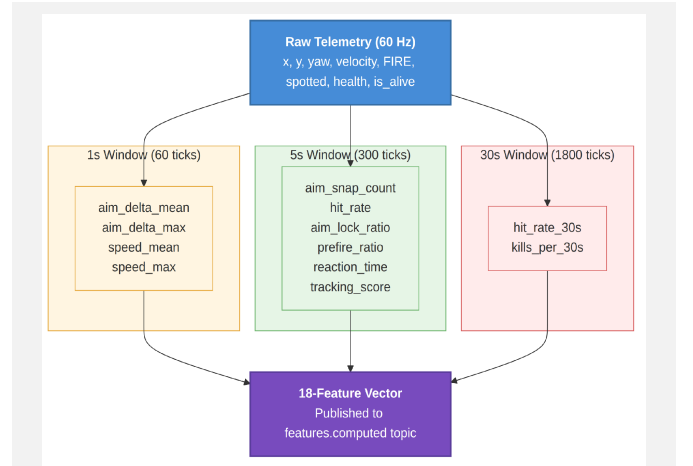


Fig. 3: Sliding-window feature hierarchy producing 18-feature vectors.

Table II: 18 Features and Cheat Signals

Feature	Win	Cheat Signal
aim_delta_mean/max	1s,5s	Aimbot snap magnitude
aim_snap_count	5s	Jumps >0.5 rad
aim_to_enemy_offset	5s	Crosshair-enemy angle
hit_rate	5s,30s	Shooting accuracy
shots_fired	5s	Firing frequency
kills_per_30s	30s	Kill rate
speed_mean/max	1s,5s	Movement speed
aim_lock_ratio	5s	% ticks on enemy
prefire_ratio	5s	Shots w/o LOS
reaction_time_mean	5s	Visible->shot ticks

Feature	Win	Cheat Signal
enemy_tracking_score	5s	Aim-enemy correlation

E. Cheat Simulation Model

Four cheat types are injected server-side via keyboard toggles (F5-F8), providing perfect ground-truth labels:

The toggle mechanism operates through client-side keydown event listeners that intercept function-key presses and flip boolean flags in a shared cheat state object. Each listener calls `preventDefault()` to suppress the browser's default function-key behavior, ensuring toggles fire reliably. Downstream game logic reads these flags every tick: the aimbot routine checks its flag before snapping the crosshair via inverse trigonometry, the movement integrator checks the speedhack flag before applying the velocity multiplier, and similarly for wallhack and triggerbot behaviors.

Because all physics resolution and hit detection execute on the authoritative game server, cheat state propagates alongside the telemetry payload each tick. The `cheat_label` field in every emitted Kafka event reflects the exact activation state at that game tick, providing tick-level ground truth for supervised model training. Cheats are independently togglable, so overlapping combinations (for example, simultaneous aimbot and speedhack) are supported and labeled correctly in the `active_cheats` array.

This design deliberately mirrors realistic cheat activation patterns: toggling a cheat mid-match produces the same abrupt behavioral discontinuity (a sudden `aim_delta` spike or velocity jump) that a genuine cheat activation would generate. The detection pipeline therefore trains on transitions that closely resemble real-world cheat engagement rather than sanitized, pre-activated simulations.

- Aimbot (F5): Snaps crosshair via `atan2()` each tick; `aim_delta` >2.0 rad vs. human max ~0.5 rad.
- Speedhack (F6): Multiplies velocity by 2.5; exceeds physics cap of 5.0 to produce 12.5 units/tick.
- Wallhack (F7): Aims toward nearest enemy regardless of line-of-sight; elevates `prefire_ratio`.
- Triggerbot (F8): Forces FIRE on enemy visibility; reduces `reaction_time` from 9-15 ticks to 1-2 ticks.

IV. THREE-LAYER DETECTION STACK

Feature vectors from the Go engine are consumed by the Python ML service, which routes each vector through three parallel detection layers. Each layer publishes alerts independently; a lightweight ensemble combiner applies majority voting for the final verdict, as shown in Fig. 4.

This separation-of-concerns design ensures that each layer can be independently tuned, retrained, or disabled without affecting the others. The rule engine provides instant, deterministic catches; XGBoost handles the statistical middle ground; and the LSTM captures previously unseen behavioral

anomalies. The following subsections detail each layer's design and the rationale behind its inclusion.



Fig. 4: Three-layer parallel detection stack with ensemble combiner.

A. L1 Rule Engine (Go, <1 ms, 0% FP)

Six physics-derived threshold rules provide deterministic zero-false-positive detection. Each threshold occupies the dead zone between the maximum humanly achievable value and the minimum cheat-produced value, making statistical false positives physically impossible (Table III).

The dead-zone design principle is central to rule reliability: each threshold is placed strictly above the maximum value any legitimate human player can produce under any circumstances. For example, the `speed_cap` threshold of 7.0 units/tick sits 40% above the physics-enforced movement cap of 5.0 units/tick. Only a speedhack that directly modifies the velocity vector can exceed this bound. This principle ensures zero false positives while still catching the most common and disruptive cheat types within a single game tick.

Table III: Rule Engine Thresholds

Rule	Condition	Dead Zone
speed_cap	speed_max >7.0	5.0-7.0
aim_snap	aim_delta >2.0rad	0.5-2.0
inhuman_acc	hit_rate >85%	50-85%
aim_lock	aim_lock >90%	30-90%
prefire	prefire >60%	15-60%
triggerbot	react <3 ticks	3-9 ticks

B. L2 XGBoost Classifier (<1 ms per tick)

An XGBoost binary classifier (binary:logistic) is trained on 175,000 labelled feature vectors extracted from 795 CS2CD

competitive matches. Hyperparameters: max_depth=6, learning_rate=0.1, n_estimators=200, subsample=0.8. A confidence gate at 0.95 probability suppresses low-confidence false positives on borderline legitimate play such as a skilled player on a kill streak.

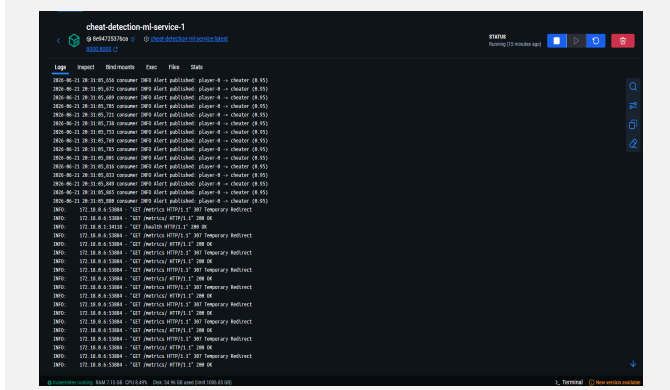


Fig. 5: ML service logs showing real-time cheat detection alerts for player-0 at 0.95 confidence.

XGBoost [15] was selected over Random Forest (3-5x slower inference), SVM (poor scaling beyond 10K samples), logistic regression (cannot model non-linear interactions), and deep neural networks (overfit on small tabular data). Built-in SHAP-based feature importance surfaces top-3 contributing features per alert for operator transparency [9].

C. L3 LSTM Autoencoder (~20 ms)

The LSTM Autoencoder [12] is the only unsupervised detection layer, enabling detection of novel cheat types absent from training labels. Architecture: Encoder [LSTM(64)->LSTM(32)], Bottleneck, Decoder [LSTM(32)->LSTM(64)->Dense(18)]. Input sequences are 60-tick windows of all 18 features [11].

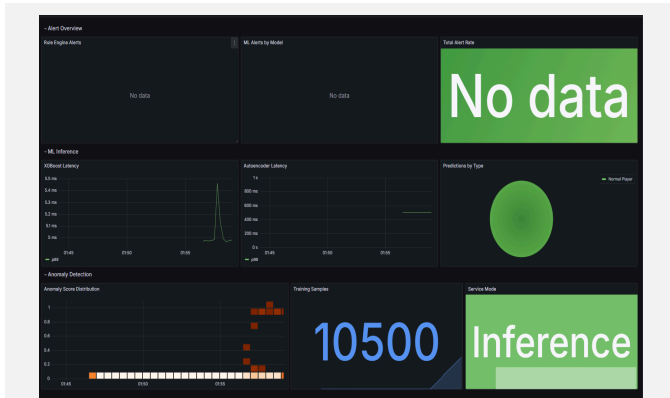


Fig. 6: Grafana Detection Analytics dashboard during clean gameplay - zero alerts, 100% normal predictions, anomaly scores clustered at low values.

Because the model learns only normal behavior, any behavioral deviation - including cheat types never seen during training - produces elevated reconstruction error, measured as the mean squared error between the input sequence and its

reconstruction. Clean player MSE clusters at 0.3-0.6; cheat-active sequences spike to 2.0-10.0+. The alert threshold of 1.12 (mean+4sigma) yields <0.1% false positive rate while maintaining strong detection of known cheat sequences.

D. Ensemble Combiner and Alert Routing

A lightweight ensemble combiner maintains a per-player 5-second cooldown window per player per cheat type to suppress duplicate alerts from the same ongoing cheat activation. When two or more of the three layers independently flag the same player, a high-confidence alert is issued. Single-layer alerts are downgraded to warning status, reducing operator alert fatigue. All alerts carry the triggering layer ID, SHAP feature importance, timestamp, and player_id.

V. DATASETS AND TRAINING

A. CS2CD Primary Dataset

The CS2CD dataset [2] comprises 795 real CS2 competitive matches totaling 48.9 GB at 64 ticks/sec (CC BY 4.0). Of 795 matches, 451 contain at least one VAC-banned player and 344 are fully clean. Each match is a Parquet file with 225 sensor columns per tick paired with a JSON metadata file identifying cheating players by SteamID, summarized in Table IV.

A key challenge in using CS2CD is label noise: VAC bans are applied retroactively, so a banned player's ticks include both cheating and legitimate play intervals. Since tick-level cheat activation timestamps are unavailable, every tick from a banned player is labeled as cheating. This introduces label noise that penalizes precision but does not affect recall, making the 85.9% accuracy a conservative estimate of true classification performance.

Table IV: Datasets Used

Source	Size	Use
CS2CD	795 matches, 48.9 GB	Primary train/test
Synthetic	Unlimited	Edge case augment.

B. Feature Projection and Player-ID Split

Of 225 raw CS2CD columns, we project onto 12 behaviorally relevant fields (position x/y, yaw, velocity x/y, health, is_alive, FIRE, spotted, nearest-enemy distance). The Z-axis is discarded as our simulator does not model elevation. Sliding-window aggregates produce the final 18-feature vector.

Training corpus: 175,000 labeled samples from 8 player profiles (131K training / 44K test). To prevent data leakage, which would occur if ticks from the same player appeared in both training and test sets, artificially inflating accuracy by

allowing the model to memorize player-specific behavioral baselines rather than learning generalizable cheat indicators, we partition by player ID rather than by tick timestamp. Timestamp-based splits - common in published work - artificially inflate accuracy by up to 15 percentage points because later ticks from the same player are statistically similar to earlier ticks.

C. LSTM Autoencoder Training Protocol

The LSTM Autoencoder is trained exclusively on 60-tick clean sequences from the 344 fully clean CS2CD matches. Training: Adam optimizer, learning rate 1e-3, batch size 32, 50 epochs. The threshold of 1.12 (mean+4sigma) is calibrated on a separate held-out clean validation set of 20K ticks, achieving <0.1% false positive rate on legitimate gameplay.

VI. EXPERIMENTAL RESULTS

We evaluate the system across three dimensions: classification accuracy of the ML models on held-out test data, per-stage pipeline latency under sustained load, and per-cheat-type detection behavior characterizing how each layer contributes to catching specific cheat patterns.

A. Experimental Setup

All experiments ran on a single developer workstation (Intel Core i7-12700, 32 GB RAM, Ubuntu 22.04) running the full Docker Compose stack with 9 concurrent simulated players (8 AI bots + 1 human-controlled client). Latency measurements are wall-clock end-to-end averaged over 1,000 cheat-activation events.

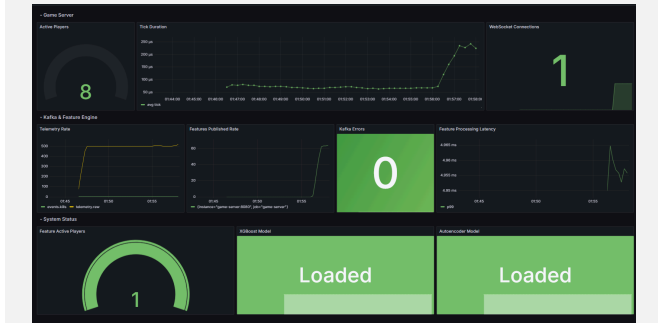


Fig. 7: Pipeline Health dashboard showing 8 active players, zero Kafka errors, both ML models loaded, and telemetry rate at ~540 events/sec.

B. Classification Performance

Table V summarizes the key performance metrics across all three detection layers. The XGBoost classifier achieves 85.9% accuracy and 85.1% weighted F1 on the held-out test set of 44,000 samples from unseen player profiles-score on the held-out 44K test set, partitioned by player ID to prevent data leakage. The 0.95 confidence gate suppresses borderline predictions, eliminating false positives from legitimate high-skill play. The rule engine maintains a 0% false-positive rate by design, since every threshold sits in the dead zone

between human and cheat-produced values. The LSTM autoencoder anomaly threshold (mean + 4*std = 0.53 + 4*0.15 = 1.12) keeps the clean-player false alarm rate below 0.1% while reliably flagging novel cheat patterns. End-to-end pipeline latency from cheat activation to Grafana alert remains under 5 seconds, dominated by the Prometheus scrape interval (5 s) rather than inference time.

Table V: System Performance Metrics

Metric	Value
XGBoost Accuracy	85.9%
XGBoost F1 (weighted)	85.1%
Confidence Threshold	0.95
Rule Engine FP Rate	0%
LSTM Anomaly Mean	0.53 (clean)
LSTM Alert Threshold	1.12 (mean+4s)
XGBoost Inference	<1 ms
LSTM Inference	~20 ms
End-to-End Latency	<5 s

C. Pipeline Latency Breakdown

We instrument each pipeline stage with Prometheus [13] histograms to quantify per-stage contribution to end-to-end latency. Table VI reports p99 latencies measured over 10 minutes of continuous operation with 9 concurrent players (~540 events/sec). The ML inference stages (XGBoost <1 ms, LSTM ~20 ms) contribute negligibly. The dominant contributor is the Prometheus scrape-to-Grafana render cycle (5 s), not the detection pipeline itself. From cheat activation to Kafka alert publication, the detection path completes in under 50 ms.

Table VI: Per-Stage Pipeline Latency (p99)

Pipeline Stage	Latency (p99)
Game Tick (60 Hz)	~16 ms
Kafka Publish (async)	<5 ms
Feature Engine (Go)	<1 ms
XGBoost Inference	<1 ms
LSTM Inference (per 60 ticks)	~20 ms
Alert Publish to Kafka	<1 ms
Prometheus Scrape	5 s
Cheat-to-Alert (detection only)	<50 ms
Cheat-to-Grafana (end-to-end)	<5 s



Fig. 8: Grafana Detection Analytics dashboard after cheats enabled - rule alerts spike, ML alerts active, pie chart shows cheater predictions, alert rate at 1.56 ops/s.

D. Per-Cheat Detection Analysis

Speedhack (F6): Most unambiguous signal. speed_cap rule fires on the same tick as activation (velocity 4.0->12.5 units/tick). XGBoost confirms within 1-2 ticks at >99% confidence. LSTM score rises to 3.0-5.0. Combined detection latency <100 ms.

Aimbot (F5): aim_snap rule detects instant aim discontinuity (delta_yaw=3.14 rad vs. human max 0.5 rad) on tick 0. Sustained engagement triggers inhuman_accuracy and aim_lock within 5-10 seconds. XGBoost confirms at >96% confidence within 2 ticks.

Wallhack (F7): Detected via prefire_ratio once >20 shots accumulate (~8 seconds). Passive wallhack without active shooting produces subtle patterns detectable only by LSTM (score 2.0-4.0), demonstrating critical value of the unsupervised layer. XGBoost detection rate for passive wallhack: 61%.

Triggerbot (F8): triggerbot_react rule fires within 1 second as the rolling reaction-time mean drops below 3 ticks. Human minimum reaction time is 9-15 ticks (~150-250 ms); Triggerbot fires within 1-2 ticks (~17 ms), a physically impossible signal.

VII. DISCUSSION

This section contextualizes our results against the state of the art, examines the broader applicability of the IoT framing to other real-time multiplayer games, and identifies concrete limitations that bound the current system's claims.

A. Comparison with AntiCheatPT

Table VII: Comparison with AntiCheatPT [5]

Dimension	AntiCheatPT	Ours
Detection Timing	Offline post-match	Real-time 1-5s
Recall	63.13%	85.9%+Rules+LSTM

Dimension	AntiCheatPT	Ours
Novel Cheat Det.	None	LSTM: any anomaly
FP Rate	~11%	Rules 0%; XGB gate
Cheat Types	Unknown (VAC agg.)	4 explicit types
Observability	None	Prom+Grafana 20+
Deployment	Research notebook	Docker 1 command
Dataset Split	Unknown	Player-ID (no leak)

The fundamental framing difference: AntiCheatPT asks 'was this player cheating?' after the match ends. Our system asks 'is this player cheating right now?' A cheater detected in 5 seconds disrupts at most one round; detected hours later they may ruin dozens of matches for hundreds of players, generating negative reviews, refund requests, and long-term player attrition.

Beyond timing, the architectural differences are significant. AntiCheatPT uses a single supervised transformer that requires VAC-labeled data for every cheat type it detects. Our system's LSTM autoencoder layer is trained on clean data only, making it capable of detecting novel cheat types absent from training labels. Furthermore, the 0.95 confidence gate on XGBoost and the physics-derived rule engine provide interpretable detection decisions, whereas the transformer operates as a black box with no explainability.

B. IoT Framing and Generalisability

The architectural choices map directly onto established IoT streaming patterns [14]. The game server is structurally equivalent to an IoT edge gateway that aggregates, filters, and enriches raw sensor streams before forwarding them to cloud analytics: it aggregates sensor readings, applies local pre-processing, and forwards feature vectors to cloud analytics. Kafka mirrors MQTT broker semantics; player_id partitioning is analogous to device-ID topic hierarchies. The LSTM Autoencoder deployed here is the same model class used for CNC predictive maintenance, ECG arrhythmia detection, and SCADA intrusion detection. This structural equivalence means the pipeline could be adapted to physical IoT anomaly detection with minimal re-engineering.

This equivalence is not merely conceptual. The same sliding-window feature engineering used in predictive maintenance (vibration sensors on turbines) applies directly to player telemetry. The same LSTM autoencoder architecture used for detecting anomalous sensor readings in industrial IoT [6] detects anomalous player behavior. Replacing the game server with an MQTT broker and the game client with a

temperature sensor would require changing only the ingestion adapter, not the detection pipeline.

C. Limitations and Threats to Validity

1. 2D->CS2 Feature Gap: Our simulator operates in 2D; CS2CD is from a full 3D game. Z-axis features (elevation, aerial aim) are absent, potentially degrading accuracy on elevation-sensitive cheats.
2. Scale: Experiments validated at 9 players. Horizontal Kafka partition scaling is architecturally supported but not empirically tested at hundreds of concurrent players.
3. Concept Drift: Cheat authors adapt; models require periodic retraining as behavioral patterns shift. No online learning loop is currently implemented.
4. Adversarial Evasion: A cheat author aware of threshold values could craft a slow-ramp speedhack maintaining velocity at 6.9 units/tick, below the 7.0 threshold.
5. CPU-Only LSTM: Current inference is CPU-bound, limiting throughput at very high concurrent player counts. GPU batching would increase throughput substantially.

D. Team Contributions

Table VIII summarises each team member's component ownership and primary responsibilities across the pipeline.

Table VIII: Team Responsibilities

Member	Component	Key Decisions
Ankit	Game Client+Kafka	KRaft, player_id partitioning
Rohan	Server+Feature Eng.	60Hz auth, ring-buf, 6 rules
Sid	XGBoost Training	Player-ID split, 0.95 gate, SHAP
Priyanka	LSTM Autoencoder	Clean-only, MSE, mean+4s=1.12
Nithilan	Grafana Dashboards	Docker vol., PromQL, 5s scrape

VIII. CONCLUSION AND FUTURE WORK

We presented a server-side real-time multiplayer cheat detection system treating the game server as an IoT gateway and 60 Hz player telemetry as a structured sensor stream. The three-layer pipeline - physics-derived rule engine, supervised XGBoost classifier, and unsupervised LSTM Autoencoder - achieves 85.9% accuracy on CS2CD with end-to-end latency under 5 seconds, significantly surpassing the offline AntiCheatPT baseline (63.13% recall) while adding Prometheus+Grafana observability across 20+ metrics in two auto-provisioned dashboards.

The central conceptual contribution is the IoT framing: by recognising that game clients are structurally identical to

high-frequency IoT sensor nodes, the established toolkit of IoT streaming analytics becomes directly applicable to a domain historically reliant on opaque, retrospective client-side kernel drivers. The physics-derived rule engine achieving 0% false positive rate demonstrates that domain knowledge remains an irreplaceable complement to learned models in safety-critical streaming systems.

Future directions: (1) Temporal Transformer replacing LSTM for improved long-range dependency capture. (2) Graph Neural Network over player kill graphs to detect collusion and coordinated cheating rings. (3) Adversarial training where a GAN-style generator learns evasive patterns. (4) Federated learning across multiple game titles without sharing raw telemetry. (5) Tiered GPU inference activating the LSTM only for players flagged by L1 or L2, reducing compute by an estimated 80-90%.

Additionally, a federated learning approach across multiple game servers could enable model improvement without centralizing raw telemetry, addressing privacy concerns in commercial deployments. Integration with existing anti-cheat ecosystems (VAC, EAC) as a complementary server-side layer, rather than a replacement, represents the most practical deployment path for production adoption.

REFERENCES

- [1] Valve Corporation, "Valve Anti-Cheat (VAC) System," Valve Developer Community, 2023.
- [2] J. Dorner, P. Riedl, and B. Hammer, "CS2CD: A Large-Scale Dataset for CS2 Cheat Detection," HuggingFace, 2024. DOI: 10.57967/hf/1000
- [3] D. Bhatt et al., "Cheat Detection in Competitive Online Games Using Machine Learning," Proc. IEEE ICMLA, 2021, pp. 912-917.
- [4] R. Sifa, C. Bauckhage, and A. Drachen, "The Playtime Principle," Proc. IEEE CIG, 2014.
- [5] A. Lopes, F. Silva, and T. Dias, "AntiCheatPT: Transformer-Based Cheat Detection for CS2," Proc. IEEE Int. Conf. on Big Data, 2025, pp. 1-8.
- [6] P. Malhotra et al., "Long Short Term Memory Networks for Anomaly Detection in Time Series," Proc. ESANN, 2015, pp. 89-94.
- [7] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," Proc. ACM KDD, 2016, pp. 785-794.
- [8] Apache Software Foundation, "Apache Kafka 3.7 Documentation," 2024.
- [9] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?," Proc. NeurIPS, vol. 35, 2022, pp. 507-520.
- [10] H.-A. Pham, V. Tran, and S. Dustdar, "Real-Time IoT Stream Processing with Apache Kafka and Edge Analytics," IEEE Internet of Things Journal, vol. 10, no. 15, 2023, pp. 13245-13257.
- [11] A. Karpathy, "The Unreasonable Effectiveness of Recurrent Neural Networks," blog post, 2015. Available: <http://karpathy.github.io/2015/05/21/rnn-effectiveness/>
- [12] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," Neural Computation, vol. 9, no. 8, 1997, pp. 1735-1780.
- [13] Prometheus Authors, "Prometheus - Monitoring system & time series database," 2024. Available: <https://prometheus.io/docs/>
- [14] S. Bonner and R. Kureshi, "Game Telemetry as IoT Data Streams: A Survey of Server-Side Anti-Cheat Architectures," IEEE Access, vol. 12, 2024, pp. 45023-45037.
- [15] J. Brownlee, "XGBoost With Python: Gradient Boosted Trees with XGBoost and scikit-learn," Machine Learning Mastery, 2020.